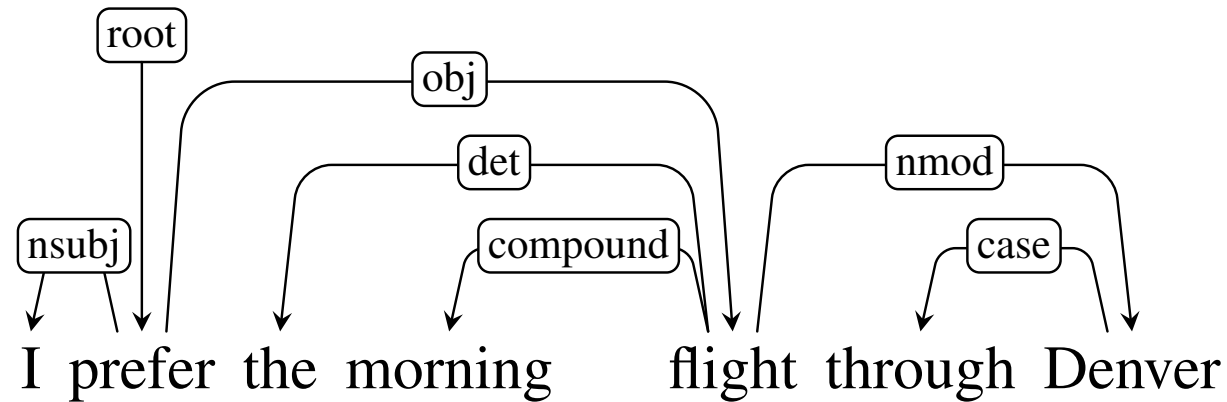


Dependency Parsing

- In dependency formalisms, phrasal constituents and phrase-structure rules do not play a direct role. Instead, the syntactic structure of a sentence is described solely in terms of directed binary grammatical relations between the words, as in the following dependency parse:



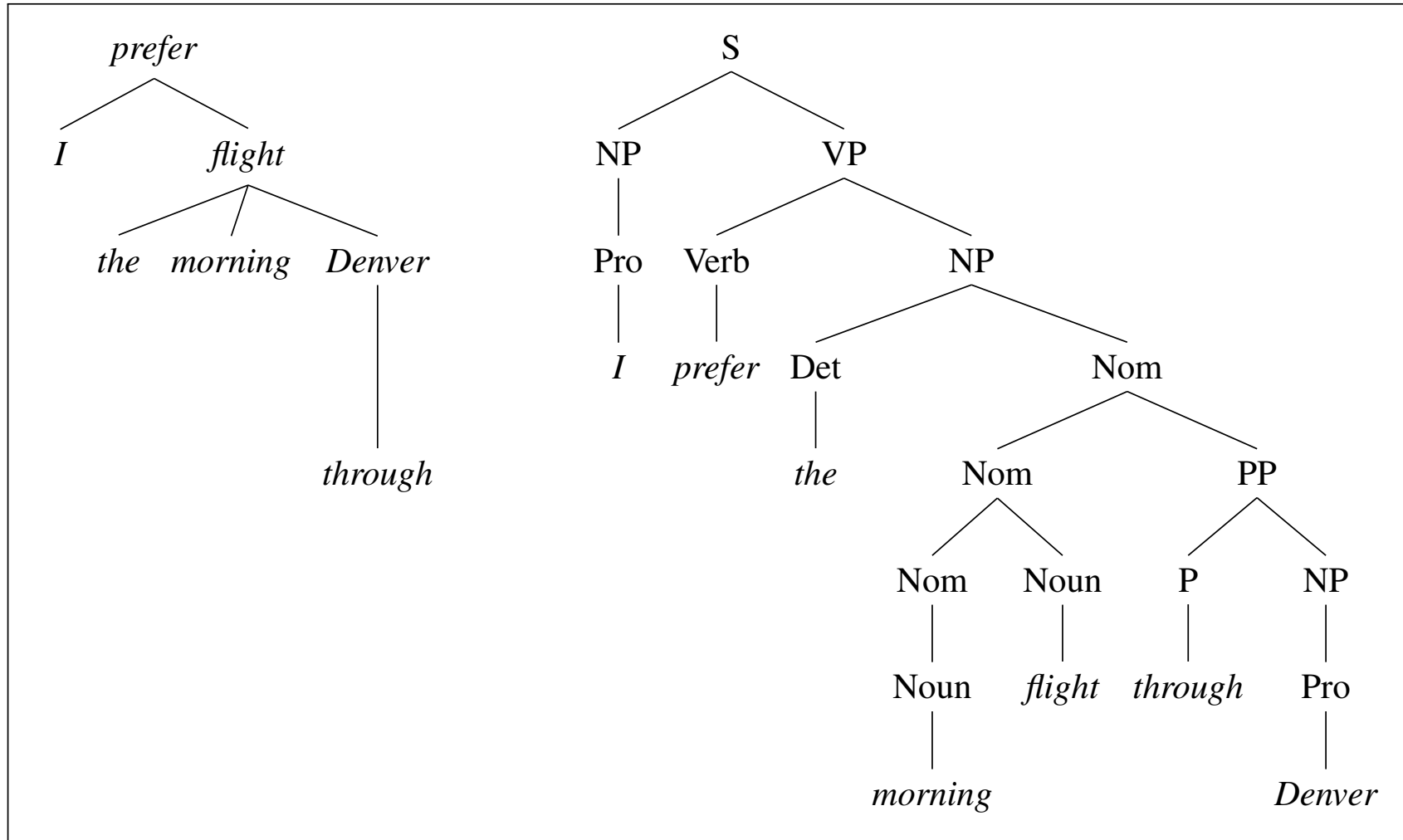
- Relations among the words are illustrated above the sentence with directed, labelled arcs from heads to dependents. We call this a **typed dependency structure** because the labels are drawn from a fixed inventory of grammatical relations. A root node explicitly marks the root of the tree, the head of the entire structure.

Advantage of Dependency Parsing

- These head-dependent relationships directly encode important information that is often buried in the more complex phrase-structure parses.
- For example, the arguments to the verb prefer are directly linked to it in the dependency structure, while their connection to the main verb is more distant in the phrase-structure tree.
- Similarly, morning and Denver, modifiers of flight, are linked to it directly in the dependency structure.
- This fact that the head-dependent relations are a good proxy for the semantic relationship between predicates and their arguments is an important reason why dependency grammars are currently more common than constituency grammars in natural language processing.

- Another major advantage of dependency grammars is their ability to deal with languages that have a relatively free word order.
- For example, word order in Czech can be much more flexible than in English; a grammatical object might occur before or after a location adverbial.
- A phrase-structure grammar would need a separate rule for each possible place in the parse tree where such an adverbial phrase could occur.

Dependency and constituent analyses for I prefer the morning flight through Denver.



Dependency Relations

- The traditional linguistic notion of grammatical relation provides the basis for the binary relations that comprise these dependency structures.
- The arguments to these relations consist of a head and a dependent.
- The head plays the role of the central organizing word, and the dependent as a kind of modifier.
- The head-dependent relationship is made explicit by directly linking heads to the words that are immediately dependent on them.

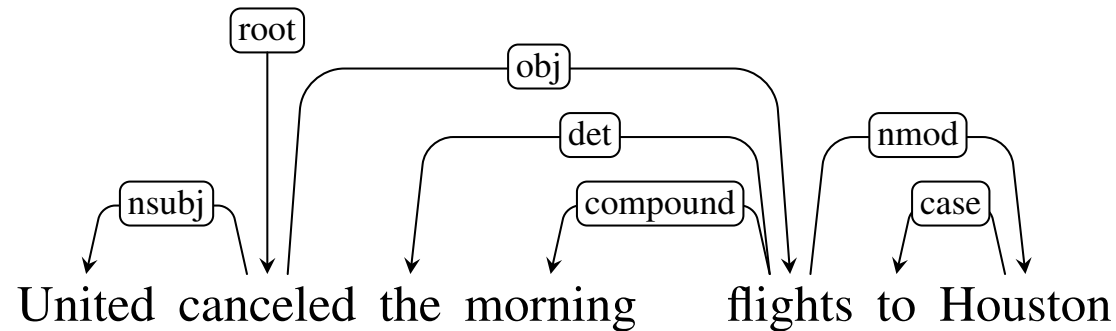
- Linguists have developed taxonomies of relations that go well beyond the familiar notions of subject and object.
- The Universal Dependencies (UD) project (de Marneffe et al., 2021), an open community effort to annotate dependencies and other aspects of grammar across more than 100 languages, provides an inventory of 37 dependency relations.

Clausal Argument Relations	Description
NSUBJ	Nominal subject
OBJ	Direct object
IOBJ	Indirect object
CCOMP	Clausal complement
Nominal Modifier Relations	Description
NMOD	Nominal modifier
AMOD	Adjectival modifier
APPOS	Appositional modifier
DET	Determiner
CASE	Prepositions, postpositions and other case markers
Other Notable Relations	Description
CONJ	Conjunct
CC	Coordinating conjunction

Examples of some Universal Dependency relations.

Relation	Examples with <i>head</i> and dependent
NSUBJ	United <i>canceled</i> the flight.
OBJ	United <i>diverted</i> the flight to Reno. We <i>booked</i> her the first flight to Miami.
IOBJ	We <i>booked</i> her the flight to Miami.
COMPOUND	We took the morning <i>flight</i> .
NMOD	<i>flight</i> to Houston .
AMOD	Book the cheapest <i>flight</i> .
APPOS	<i>United</i> , a unit of UAL, matched the fares.
DET	The <i>flight</i> was canceled. Which <i>flight</i> was delayed?
CONJ	We <i>flew</i> to Denver and drove to Steamboat.
CC	We flew to Denver and <i>drove</i> to Steamboat.
CASE	Book the flight through <i>Houston</i> .

- The core set of frequently used relations can be broken into two sets:
 - clausal relations that describe syntactic roles with respect to a predicate (often a verb),
 - and modifier relations that categorize the ways that words can modify their heads.
- Consider, for example, the following sentence:



- Here the clausal relations NSUBJ and OBJ identify the subject and direct object of the predicate cancel,
- while the NMOD, DET, and CASE relations denote modifiers of the nouns flights and Houston

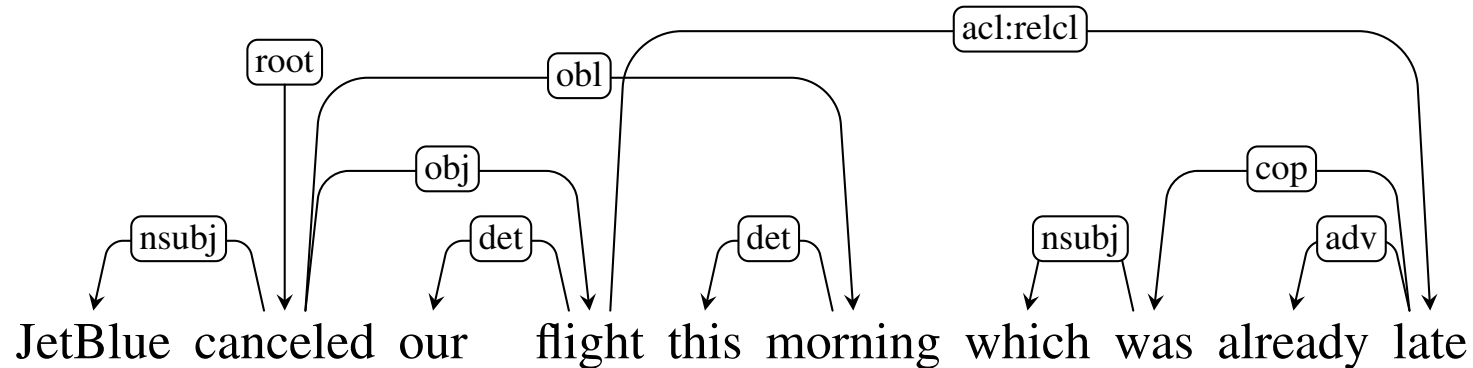
- In dependency parsing, a **clausal relation** links a word to something that acts as part of the clause's core predicate structure, while a **modifier relation** links a word that adds extra information like time, place, manner, or description.
- She read the book **yesterday**.
- She → nsubj of read (Clausal relation)
- book → dobj of read (Clausal relation)
- yesterday → advmod of read. (modifier relation)

Dependency Formalisms

- A dependency structure can be represented as a directed graph $G = (V, A)$, consisting of a set of vertices V , and a set of ordered pairs of vertices A , which we'll call arcs.
- We will assume that the set of vertices, V , corresponds exactly to the set of words in a given sentence.
- A dependency tree is a directed graph that satisfies the following constraints:
 1. There is a single designated root node that has no incoming arcs.
 2. With the exception of the root node, each vertex has exactly one incoming arc.
 3. There is a unique path from the root node to each vertex in V .

Projectivity

- An arc from a head to a dependent is said to be projective if there is a path from the head to every word that lies between the head and the dependent in the sentence.
- A dependency tree is then said to be projective if all the arcs that make it up are projective.



- In this example, the arc from flight to its modifier late is non-projective since there is no path from flight to the intervening words this and morning.
- A dependency tree is projective if it can be drawn with no crossing edges.

Why checking for Projectivity?

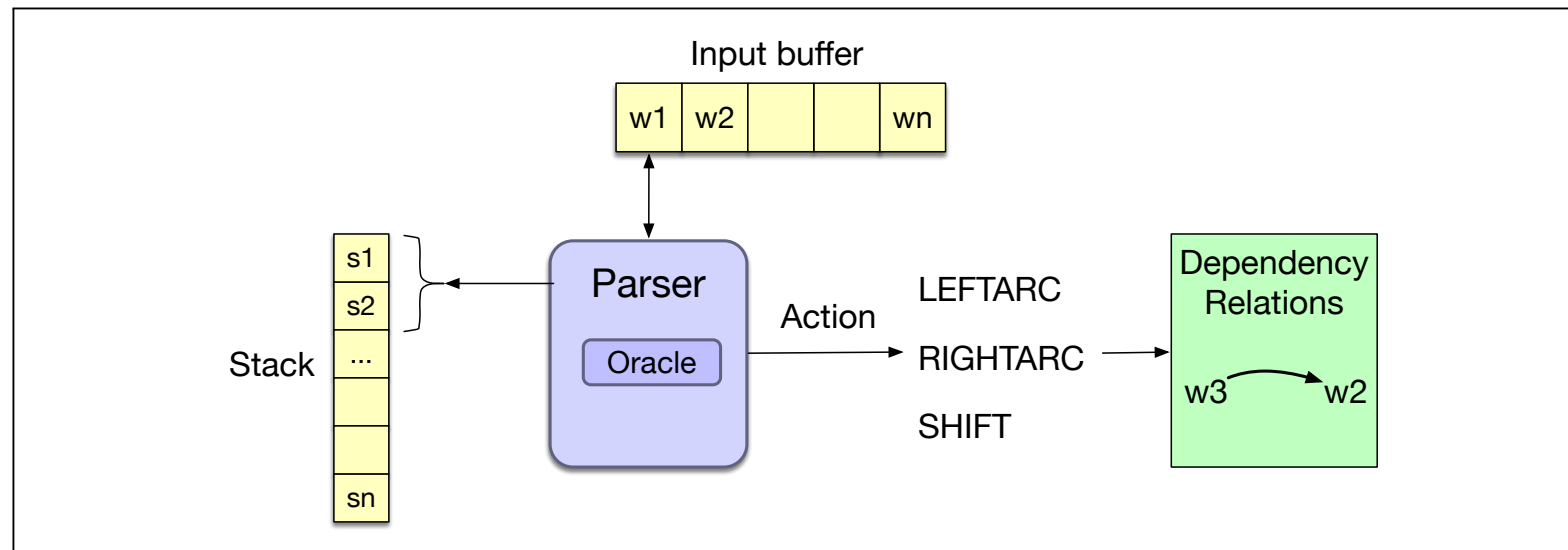
- The most widely used English dependency treebanks were automatically derived from phrase-structure treebanks through the use of head-finding rules.
- The trees generated in such a fashion will always be projective, and hence will be incorrect when non-projective examples like this one are encountered.
- The transition-based approaches can only produce projective trees, hence any sentences with non-projective structures will necessarily contain some errors.

Dependency Treebanks

- Treebanks play a critical role in the development and evaluation of dependency parsers.
- They are used for training parsers, they act as the gold labels for evaluating parsers, and they also provide useful information for corpus linguistics studies.
- Dependency treebanks are created by having human annotators directly generate dependency structures for a given corpus, or by hand-correcting the output of an automatic parser.
- A few early treebanks were also based on using a deterministic process to translate existing constituent-based treebanks into dependency trees.

Transition-Based Dependency Parsing

- Our first approach to dependency parsing is called transition-based parsing.
- This architecture draws on shift-reduce parsing,
- In transition-based parsing we'll have a stack on which we build the parse, a buffer of tokens to be parsed, and a parser which takes actions on the parse via a predictor called an oracle



- The parser walks through the sentence left-to-right, successively shifting items from the buffer onto the stack.
- At each time point we examine the top two elements on the stack, and the oracle makes a decision about what transition to apply to build the parse.
- The possible transitions correspond to the intuitive actions one might take in creating a dependency tree by examining the words in a single pass over the input from left to right
 - Assign the current word as the head of some previously seen word,
 - Assign some previously seen word as the head of the current word,
 - Postpone dealing with the current word, storing it for later processing.

- LEFTARC: Assert a head-dependent relation between the word at the top of the stack and the second word; remove the second word from the stack.
- RIGHTARC: Assert a head-dependent relation between the second word on the stack and the word at the top; remove the top word from the stack;
- SHIFT: Remove the word from the front of the input buffer and push it onto the stack.

Trace of a transition-based parse

Step	Stack	Word List	Action	Relation Added
0	[root]	[book, me, the, morning, flight]	SHIFT	(book → me)
1	[root, book]	[me, the, morning, flight]	SHIFT	
2	[root, book, me]	[the, morning, flight]	RIGHTARC	
3	[root, book]	[the, morning, flight]	SHIFT	
4	[root, book, the]	[morning, flight]	SHIFT	
5	[root, book, the, morning]	[flight]	SHIFT	(morning ← flight) (the ← flight) (book → flight) (root → book)
6	[root, book, the, morning, flight]	[]	LEFTARC	
7	[root, book, the, flight]	[]	LEFTARC	
8	[root, book, flight]	[]	RIGHTARC	
9	[root, book]	[]	RIGHTARC	
10	[root]	[]	Done	

Creating an Oracle

- The oracle for greedily selecting the appropriate transition is trained by supervised machine learning.
- As with all supervised machine learning methods, we will need training data: configurations annotated with the correct transition to take.
- We can draw these from dependency trees.

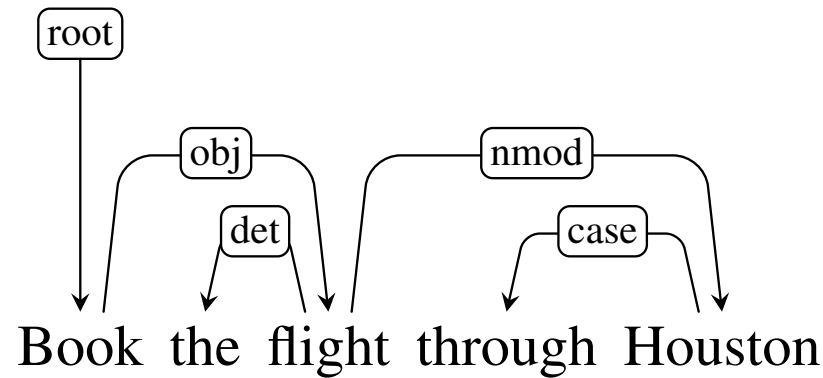
Generating Training Data

- The oracle from the algorithm takes as input a configuration and returns a transition operator.
- Therefore, to train a classifier, we will need configurations paired with transition operators (i.e., LEFTARC, RIGHTARC, or SHIFT).
- Unfortunately, treebanks pair entire sentences with their corresponding trees, not configurations with transitions.
- To generate the required training data, we employ the oracle-based parsing algorithm in a clever way.
- We supply our oracle with the training sentences to be parsed along with their corresponding reference parses from the treebank.
- To produce training instances, we then simulate the operation of the parser by running the algorithm and relying on a new training oracle to give us correct transition operators for each successive configuration.

- Parser operation begins with a default initial configuration where the stack contains the ROOT, the input list is just the list of words, and the set of relations is empty.
- The LEFTARC and RIGHTARC operators each add relations between the words at the top of the stack to the set of relations being accumulated for a given sentence.
- Since we have a gold-standard reference parse for each training sentence, we know which dependency relations are valid for a given sentence.
- Therefore, we can use the reference parse to guide the selection of operators as the parser steps through a sequence of configurations.

- To be more precise, given a reference parse and a configuration, the training oracle proceeds as follows:
- Choose LEFTARC if it produces a correct head-dependent relation given the reference parse and the current configuration
- Otherwise, choose RIGHTARC if (1) it produces a correct head-dependent relation given the reference parse and (2) all of the dependents of the word at the top of the stack have already been assigned,
- Otherwise, choose SHIFT.

example



- At Step 1, LEFTARC is not applicable in the initial configuration since it asserts a relation, $(\text{root} \leftarrow \text{book})$, not in the reference answer;
- RIGHTARC does assert a relation contained in the final answer $(\text{root} \rightarrow \text{book})$, however book has not been attached to any of its dependents yet, so we have to defer, leaving SHIFT as the only feature possible action.
- The same conditions hold in the next two steps.
- In step 3, LEFTARC is selected to link the to its head.
- Now consider the situation in Step 4.

Stack	Word buffer	Relations
[root, book, flight]	[through, Houston]	(the $\leftarrow$ flight)

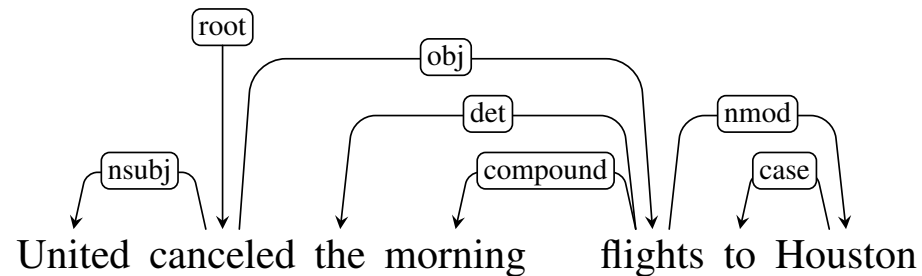
- Here, we might be tempted to add a dependency relation between book and flight, which is present in the reference parse.
- But doing so now would prevent the later attachment of Houston since flight would have been removed from the stack.
- Fortunately, the precondition on choosing RIGHTARC prevents this choice and we're again left with SHIFT as the only viable option.
- The remaining choices complete the set of operators needed for this example.

A feature-based classifier

- Feature-based classifiers generally use the same features used for part-of-speech tagging and partial parsing: Word forms, lemmas, parts of speech, the head, and the dependency relation to the head.
- The features are extracted from the training configurations, which consist of the stack, the buffer and the current set of relations.
- Most useful are features referencing the top levels of the stack, the words near the front of the buffer, and the dependency relations already associated with any of those elements.
- We'll use a feature template
- Feature templates allow us to automatically generate large numbers of specific features from a training set.

- A feature template could be
 $\langle s_1.w, op \rangle, \langle s_2.w, op \rangle \langle s_1.t, op \rangle, \langle s_2.t, op \rangle \langle b_1.w, op \rangle, \langle b_1.t, op \rangle \langle s_1.wt, op \rangle$
- Here features are denoted as location.property, where s = stack, b = the word buffer, w = word forms, t = part-of-speech, and op = operator.
- Thus the feature for the word form at the top of the stack would be $s_1.w$, the part of speech tag at the front of the buffer $b_1.t$, and the concatenated feature $s_1.wt$ represents the word form concatenated with the part of speech of the word at the top of the stack.

- Consider applying these templates to the following intermediate configuration derived from a training oracle for



Stack	Word buffer	Relations
[root, canceled, flights]	[to Houston]	(canceled → United) (flights → morning) (flights → the)

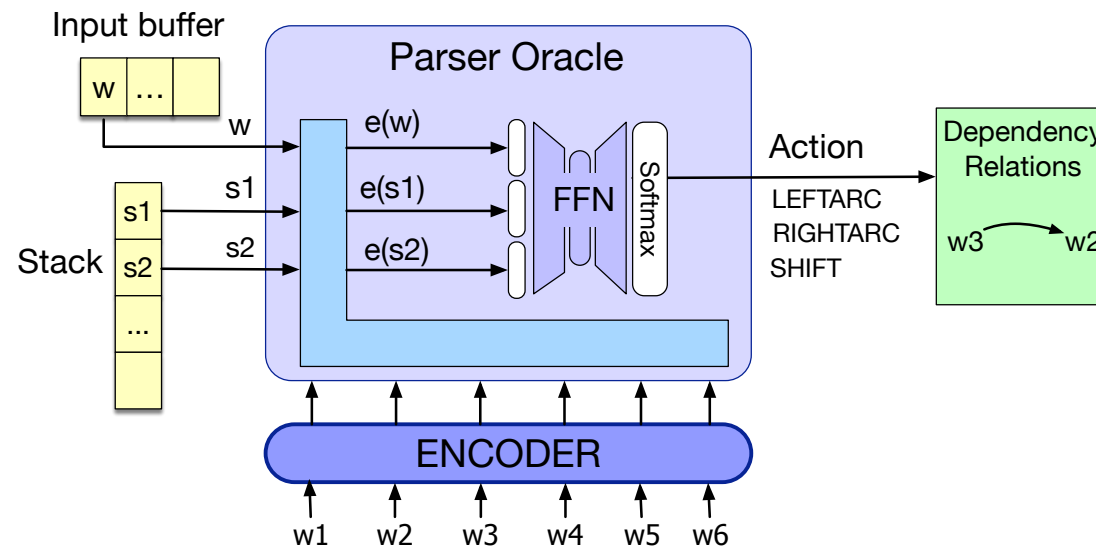
- The correct transition here is SHIFT.
- The application of our set of feature templates to this configuration would result in the following set of instantiated features.
- $\langle s_1.w = \text{flights}, \text{op} = \text{shift} \rangle \langle s_2.w = \text{canceled}, \text{op} = \text{shift} \rangle$
- $\langle s_1.t = \text{NNS}, \text{op} = \text{shift} \rangle$
- $\langle s_2.t = \text{VBD}, \text{op} = \text{shift} \rangle$
- $\langle b_1.w = \text{to}, \text{op} = \text{shift} \rangle$
- $\langle b_1.t = \text{TO}, \text{op} = \text{shift} \rangle$
- $\langle s_1.wt = \text{flightsNNS}, \text{op} = \text{shift} \rangle$

- At every step, the parser chooses a transition:
 - **SHIFT**: move first word from buffer to stack
 - **LEFT-ARC**: add arc from buffer head to stack top (and remove stack top)
 - **RIGHT-ARC**: add arc from stack top to buffer head (and move buffer head to stack)
- The goal is to simulate the correct parse tree using these transitions.
- The **correct transition** depends on:
 - The **top of the stack** (called s_1)
 - The **first word in the buffer**
 - The **gold dependency tree** (correct arcs)

- So, if no valid arc can be created between s_1 and the buffer head (according to gold tree), we should **SHIFT** to move more words onto the stack.
- In this example:
- $\langle s1.w = \text{flights}, \text{op} = \text{shift} \rangle$
- $s1.w = \text{flights} \rightarrow$ the top of the stack is the word "flights"
- The parser decides to **SHIFT** the next word from the buffer onto the stack
- That means neither a **LEFT-ARC** nor **RIGHT-ARC** between "flights" and the buffer head is allowed by the gold parse tree yet
- So we need more context (more words on the stack) $\rightarrow$ hence, **SHIFT**
- **Therefore**, the correct operation at this point is SHIFT.

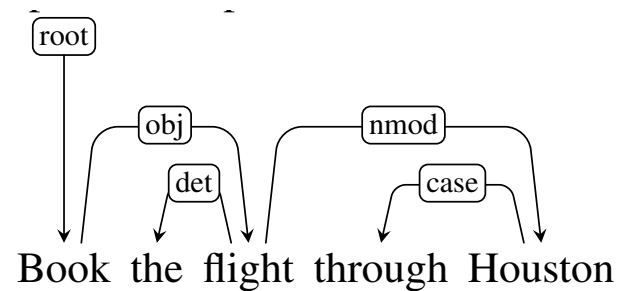
A neural classifier

- The oracle can also be implemented by a neural classifier.
- A standard architecture is simply to pass the sentence through an encoder, then take the presentation of the top 2 words on the stack and the first word of the buffer, concatenate them, and present to a feedforward network that predicts the transition to take
- Learning can be done with cross-entropy loss.



Advanced Methods in Transition-Based Parsing

- A frequently used alternative is the arc eager transition system.
- The arc eager approach gets its name from its ability to assert rightward relations much sooner than in the arc standard approach.



- Consider the dependency relation between book and flight in this analysis.
- As shown in Fig. an arc-standard approach would assert this relation at Step 8, despite the fact that book and flight first come together on the stack much earlier at Step 4.

- In an arc-standard approach, dependents are removed from the stack as soon as they are assigned their heads.
- If flight had been assigned book as its head in Step 4, it would no longer be available to serve as the head of Houston.
- The arc-eager system addresses this issue by allowing words to be attached to their heads as early as possible, before all the subsequent words dependent on them have been seen.
- This is accomplished through minor changes to the LEFTARC and RIGHTARC operators and the addition of a new REDUCE operator.

- LEFTARC: Assert a head-dependent relation between the word at the front of the input buffer and the word at the top of the stack; pop the stack.
- RIGHTARC: Assert a head-dependent relation between the word on the top of the stack and the word at the front of the input buffer; shift the word at the front of the input buffer to the stack.
- SHIFT: Remove the word from the front of the input buffer and push it onto the stack.
- REDUCE: Pop the stack.

- The LEFTARC and RIGHTARC operators are applied to the top of the stack and the front of the input buffer, instead of the top two elements of the stack as in the arc-standard approach.
- The RIGHTARC operator now moves the dependent to the stack from the buffer rather than removing it, thus making it available to serve as the head of following words.
- The new REDUCE operator removes the top element from the stack.

A processing trace of Book the flight through Houston using the arc-eager transition operators

Step	Stack	Word List	Action	Relation Added
0	[root]	[book, the, flight, through, houston]	RIGHTARC	(root → book)
1	[root, book]	[the, flight, through, houston]	SHIFT	
2	[root, book, the]	[flight, through, houston]	LEFTARC	(the ← flight)
3	[root, book]	[flight, through, houston]	RIGHTARC	(book → flight)
4	[root, book, flight]	[through, houston]	SHIFT	
5	[root, book, flight, through]	[houston]	LEFTARC	(through ← houston)
6	[root, book, flight]	[houston]	RIGHTARC	(flight → houston)
7	[root, book, flight, houston]	[]	REDUCE	
8	[root, book, flight]	[]	REDUCE	
9	[root, book]	[]	REDUCE	
10	[root]	[]	Done	

Graph-Based Dependency Parsing

- Graph-based methods are the second important family of dependency parsing algorithms.
- Graph-based parsers are more accurate than transition-based parsers, especially on long sentences
- Transition-based methods have trouble when the heads are very far from the dependents
- Graph-based dependency parsers search through the space of possible trees for a given sentence for a tree (or trees) that maximize some score.
- These methods encode the search space as directed graphs
- Formally, given a sentence S we're looking for the best dependency tree in G_s , the space of all possible trees for that sentence, that maximizes some score.

$$\hat{T}(S) = \operatorname{argmax}_{t \in G_s} \operatorname{Score}(t, S)$$

- We'll make the simplifying assumption that this score can be edge-factored, meaning that the overall score for a tree is the sum of the scores of each of the scores of the edges that comprise the tree.

$$\text{Score}(t, S) = \sum_{e \in t} \text{Score}(e)$$

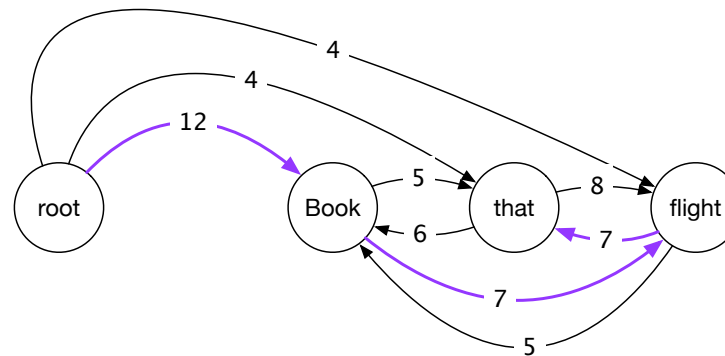
- Graph-based algorithms have to solve two problems:
 1. assigning a score to each edge, and
 2. finding the best parse tree given the scores of all potential edges.

Parsing via finding the maximum spanning tree

- In graph-based parsing, given a sentence S we start by creating a graph G which is a fully-connected, weighted, directed graph where the vertices are the input words and the directed edges represent all possible head-dependent assignments.
- We'll include an additional ROOT node with outgoing edges directed at all of the other vertices.
- The weights of each edge in G reflect the score for each possible head-dependent relation assigned by some scoring algorithm.

- It turns out that finding the best dependency parse for S is equivalent to finding the maximum spanning tree over G .
- A spanning tree over a graph G is a subset of G that is a tree and covers all the vertices in G
- A spanning tree over G that starts from the ROOT is a valid parse of S .
- A maximum spanning tree is the spanning tree with the highest score.
- Thus a maximum spanning tree of G emanating from the ROOT is the optimal dependency parse for the sentence.

- A directed graph for the example Book that flight is shown in Fig, with the maximum spanning tree corresponding to the desired parse shown in blue.



- Before describing the algorithm it's useful to consider two intuitions about directed graphs and their spanning trees. The first intuition begins with the fact that
- every vertex in a spanning tree has exactly one incoming edge.
- The second intuition is that the absolute values of the edge scores are not critical to determining its maximum spanning tree.
- Instead, it is the relative weights of the edges entering each vertex that matters.
- If we were to subtract a constant amount from each edge entering a given vertex it would have no impact on the choice of the maximum spanning tree since every possible spanning tree would decrease by exactly the same amount.

- The first step of the algorithm itself is quite straightforward. For each vertex in the graph, an incoming edge (representing a possible head assignment) with the highest score is chosen.
- If the resulting set of edges produces a spanning tree then we're done.
- Unfortunately, this approach doesn't always lead to a tree since the set of edges selected may contain cycles.
- There is a straightforward way to eliminate cycles generated during the greedy selection phase.
- Chu and Liu (1965) and Edmonds (1967) independently developed an approach that begins with greedy selection and follows with an elegant recursive cleanup phase that eliminates cycles.

- The cleanup phase begins by adjusting all the weights in the graph by subtracting the score of the maximum edge entering each vertex from the score of all the edges entering that vertex.
- Subtracting the value of the edge with maximum weight from each edge entering a vertex results in a weight of zero for all of the edges selected during the greedy selection phase, including all of the edges involved in the cycle.
- Having adjusted the weights, the algorithm creates a new graph by selecting a cycle and collapsing it into a single new node.
- Now, if we knew the maximum spanning tree of this new graph, we would have what we need to eliminate the cycle.

The Chu-Liu Edmonds algorithm for finding a maximum spanning tree in a weighted directed graph.

function MAXSPANNINGTREE($G=(V,E)$, $root$, $score$) **returns** *spanning tree*

$F \leftarrow []$

$T' \leftarrow []$

$score' \leftarrow []$

for each $v \in V$ **do**

$bestInEdge \leftarrow \operatorname{argmax}_{e=(u,v) \in E} score[e]$

$F \leftarrow F \cup bestInEdge$

for each $e=(u,v) \in E$ **do**

$score'[e] \leftarrow score[e] - score[bestInEdge]$

if $T=(V,F)$ is a spanning tree **then return** it

else

$C \leftarrow$ a cycle in F

$G' \leftarrow \text{CONTRACT}(G, C)$

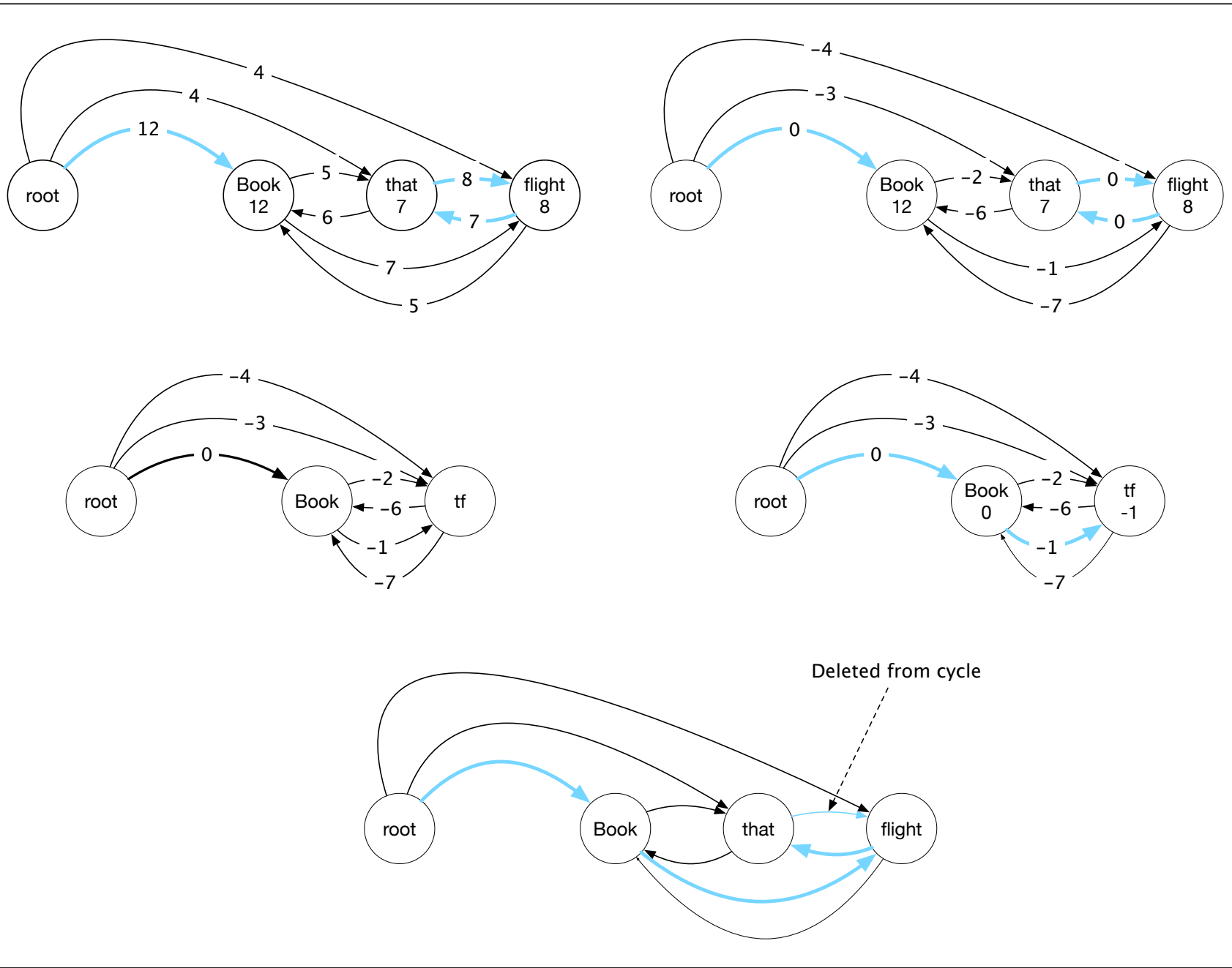
$T' \leftarrow \text{MAXSPANNINGTREE}(G', root, score')$

$T \leftarrow \text{EXPAND}(T', C)$

return T

function CONTRACT(G, C) **returns** *contracted graph*

function EXPAND(T, C) **returns** *expanded graph*



A feature-based algorithm for assigning scores

- In a feature-based algorithm we compute the edge score as a weighted sum of features extracted from it:

$$\text{score}(S, e) = \sum_{i=1}^N w_i f_i(S, e)$$

- Given this formulation, we need to identify relevant features and train the weights.
- The features (and feature combinations) used to train edge-factored models mirror those used in training transition-based parsers, such as

- Wordforms, lemmas, and parts of speech of the headword and its dependent.
- Corresponding features from the contexts before, after and between the words.
- Word embeddings.
- The dependency relation itself.
- The direction of the relation (to the right or left).
- The distance from the head to the dependent.

- Given a set of features, our next problem is to learn a set of weights corresponding to each.
- We seek to train a model that assigns higher scores to correct trees than to incorrect ones.
- An effective framework for problems like this is to use inference-based learning combined with the perceptron learning rule.
- In this framework, we parse a sentence from the training set using some initially random set of initial weights.
- If the resulting parse matches the corresponding tree in the training data, we do nothing to the weights.
- Otherwise, we find those features in the incorrect parse that are not present in the reference parse and we lower their weights by a small amount based on the learning rate.
- We do this incrementally for each sentence in our training data until the weights converge.